analyse-syndrome-grippal

November 3, 2026

1 Incidence du syndrome grippal

[1]:
```
%matplotlib inline
import os
import urllib.request
import matplotlib.pyplot as plt
import pandas as pd
import isoweek
```

Les données de l'incidence du syndrome grippal sont disponibles du site Web du Réseau Sentinelles. Nous les récupérons sous forme d'un fichier en format CSV dont chaque ligne correspond à une semaine de la période demandée. Nous téléchargeons toujours le jeu de données complet, qui commence en 1984 et se termine avec une semaine récente.

Pour ne pas dépendre d'une connexion Internet à chaque exécution, et pour garantir que l'analyse porte toujours sur les mêmes données, nous en faisons **une copie locale**: le fichier n'est téléchargé que s'il n'existe pas encore. L'URL d'origine est conservée ici pour la traçabilité: un lecteur peut retélécharger les données et les comparer à notre copie.

[2]:
```
data_url = "http://www.sentiweb.fr/datasets/incidence-PAY-3.csv"
data_file = "syndrome-grippal.csv"
```

Si le fichier local n'existe pas, nous téléchargeons les données depuis l'URL d'origine et nous les y déposons.

[3]:
```
if not os.path.exists(data_file):
    urllib.request.urlretrieve(data_url, data_file)
```

Voici l'explication des colonnes données sur le site d'origine:

Nom de colonne	Libellé de colonne
week	Semaine calendaire (ISO 8601)
indicator	Code de l'indicateur de surveillance
inc	Estimation de l'incidence de consultations en nombre de cas
inc_low	Estimation de la borne inférieure de l'IC95% du nombre de cas de consultation
inc_up	Estimation de la borne supérieure de l'IC95% du nombre de cas de consultation

Nom de colonne	Libellé de colonne
inc100	Estimation du taux d'incidence du nombre de cas de consultation (en cas pour 100,000 habitants)
inc100_low	Estimation de la borne inférieure de l'IC95% du taux d'incidence du nombre de cas de consultation (en cas pour 100,000 habitants)
inc100_up	Estimation de la borne supérieure de l'IC95% du taux d'incidence du nombre de cas de consultation (en cas pour 100,000 habitants)
geo_insee	Code de la zone géographique concernée (Code INSEE) http://www.insee.fr/fr/methodes/nomenclatures/cog/
geo_name	Libellé de la zone géographique (ce libellé peut être modifié sans préavis)

La première ligne du fichier CSV est un commentaire, que nous ignorons en précisant `skiprows=1`.

Nous lisons ensuite le fichier CSV local.

[4]:
```
raw_data = pd.read_csv(data_file, encoding='iso-8859-1', skiprows=1,␣
↪na_values=['-'])
raw_data
```

[4]:

	week	indicator	inc	inc_low	inc_up	inc100	inc100_low	\
0	202643	3	63063.0	52329.0	73797.0	94.0	78.0	
1	202642	3	57374.0	50243.0	64505.0	85.0	74.0	
2	202641	3	62204.0	55044.0	69364.0	92.0	81.0	
3	202640	3	63431.0	56318.0	70544.0	94.0	83.0	
4	202639	3	59570.0	52724.0	66416.0	88.0	78.0	
5	202638	3	51618.0	44922.0	58314.0	77.0	67.0	
6	202637	3	37015.0	31541.0	42489.0	55.0	47.0	
7	202636	3	25537.0	19707.0	31367.0	38.0	29.0	
8	202635	3	18039.0	13597.0	22481.0	27.0	20.0	
9	202634	3	12726.0	8755.0	16697.0	19.0	13.0	
10	202633	3	7188.0	3962.0	10414.0	11.0	6.0	
11	202632	3	7870.0	4582.0	11158.0	12.0	7.0	
12	202631	3	7046.0	4169.0	9923.0	10.0	6.0	
13	202630	3	8880.0	5665.0	12095.0	13.0	8.0	
14	202629	3	7601.0	4679.0	10523.0	11.0	7.0	
15	202628	3	9217.0	6138.0	12296.0	14.0	9.0	
16	202627	3	11019.0	7871.0	14167.0	16.0	11.0	
17	202626	3	11341.0	8008.0	14674.0	17.0	12.0	
18	202625	3	12022.0	8825.0	15219.0	18.0	13.0	
19	202624	3	14005.0	10335.0	17675.0	21.0	16.0	
20	202623	3	15536.0	11350.0	19722.0	23.0	17.0	
21	202622	3	11269.0	7864.0	14674.0	17.0	12.0	
22	202621	3	14005.0	10391.0	17619.0	21.0	16.0	
23	202620	3	13818.0	9441.0	18195.0	21.0	15.0	
24	202619	3	10240.0	6610.0	13870.0	15.0	10.0	
25	202618	3	11906.0	8174.0	15638.0	18.0	12.0	

26	202617		3	14808.0	10600.0	19016.0	22.0	16.0
27	202616		3	21922.0	17127.0	26717.0	33.0	26.0
28	202615		3	21286.0	16456.0	26116.0	32.0	25.0
29	202614		3	29932.0	24335.0	35529.0	44.0	36.0
…	…	…	…	…	…	…	…	
2161	198521		3	26096.0	19621.0	32571.0	47.0	35.0
2162	198520		3	27896.0	20885.0	34907.0	51.0	38.0
2163	198519		3	43154.0	32821.0	53487.0	78.0	59.0
2164	198518		3	40555.0	29935.0	51175.0	74.0	55.0
2165	198517		3	34053.0	24366.0	43740.0	62.0	44.0
2166	198516		3	50362.0	36451.0	64273.0	91.0	66.0
2167	198515		3	63881.0	45538.0	82224.0	116.0	83.0
2168	198514		3	134545.0	114400.0	154690.0	244.0	207.0
2169	198513		3	197206.0	176080.0	218332.0	357.0	319.0
2170	198512		3	245240.0	223304.0	267176.0	445.0	405.0
2171	198511		3	276205.0	252399.0	300011.0	501.0	458.0
2172	198510		3	353231.0	326279.0	380183.0	640.0	591.0
2173	198509		3	369895.0	341109.0	398681.0	670.0	618.0
2174	198508		3	389886.0	359529.0	420243.0	707.0	652.0
2175	198507		3	471852.0	432599.0	511105.0	855.0	784.0
2176	198506		3	565825.0	518011.0	613639.0	1026.0	939.0
2177	198505		3	637302.0	592795.0	681809.0	1155.0	1074.0
2178	198504		3	424937.0	390794.0	459080.0	770.0	708.0
2179	198503		3	213901.0	174689.0	253113.0	388.0	317.0
2180	198502		3	97586.0	80949.0	114223.0	177.0	147.0
2181	198501		3	85489.0	65918.0	105060.0	155.0	120.0
2182	198452		3	84830.0	60602.0	109058.0	154.0	110.0
2183	198451		3	101726.0	80242.0	123210.0	185.0	146.0
2184	198450		3	123680.0	101401.0	145959.0	225.0	184.0
2185	198449		3	101073.0	81684.0	120462.0	184.0	149.0
2186	198448		3	78620.0	60634.0	96606.0	143.0	110.0
2187	198447		3	72029.0	54274.0	89784.0	131.0	99.0
2188	198446		3	87330.0	67686.0	106974.0	159.0	123.0
2189	198445		3	135223.0	101414.0	169032.0	246.0	184.0
2190	198444		3	68422.0	20056.0	116788.0	125.0	37.0

	inc100_up	geo_insee	geo_name
0	110.0	FR	France
1	96.0	FR	France
2	103.0	FR	France
3	105.0	FR	France
4	98.0	FR	France
5	87.0	FR	France
6	63.0	FR	France
7	47.0	FR	France
8	34.0	FR	France
9	25.0	FR	France

```
10          16.0        FR   France
11          17.0        FR   France
12          14.0        FR   France
13          18.0        FR   France
14          15.0        FR   France
15          19.0        FR   France
16          21.0        FR   France
17          22.0        FR   France
18          23.0        FR   France
19          26.0        FR   France
20          29.0        FR   France
21          22.0        FR   France
22          26.0        FR   France
23          27.0        FR   France
24          20.0        FR   France
25          24.0        FR   France
26          28.0        FR   France
27          40.0        FR   France
28          39.0        FR   France
29          52.0        FR   France
…           …       …        …
2161        59.0        FR   France
2162        64.0        FR   France
2163        97.0        FR   France
2164        93.0        FR   France
2165        80.0        FR   France
2166       116.0        FR   France
2167       149.0        FR   France
2168       281.0        FR   France
2169       395.0        FR   France
2170       485.0        FR   France
2171       544.0        FR   France
2172       689.0        FR   France
2173       722.0        FR   France
2174       762.0        FR   France
2175       926.0        FR   France
2176      1113.0        FR   France
2177      1236.0        FR   France
2178       832.0        FR   France
2179       459.0        FR   France
2180       207.0        FR   France
2181       190.0        FR   France
2182       198.0        FR   France
2183       224.0        FR   France
2184       266.0        FR   France
2185       219.0        FR   France
2186       176.0        FR   France
```

```
2187      163.0        FR   France
2188      195.0        FR   France
2189      308.0        FR   France
2190      213.0        FR   France

[2191 rows x 10 columns]
```

Y a-t-il des points manquants dans ce jeux de données ? Oui, la semaine 19 de l'année 1989 n'a pas de valeurs associées.

[5]:
```
raw_data[raw_data.isnull().any(axis=1)]
```

[5]:
```
        week  indicator  inc  inc_low  inc_up  inc100  inc100_low  inc100_up  \
1954  198919          3  NaN      NaN     NaN     NaN         NaN        NaN

     geo_insee geo_name
1954        FR   France
```

Nous éliminons ce point, ce qui n'a pas d'impact fort sur notre analyse qui est assez simple.

[6]:
```
data = raw_data.dropna().copy()
data
```

[6]:

	week	indicator	inc	inc_low	inc_up	inc100	inc100_low	\
0	202643	3	63063.0	52329.0	73797.0	94.0	78.0	
1	202642	3	57374.0	50243.0	64505.0	85.0	74.0	
2	202641	3	62204.0	55044.0	69364.0	92.0	81.0	
3	202640	3	63431.0	56318.0	70544.0	94.0	83.0	
4	202639	3	59570.0	52724.0	66416.0	88.0	78.0	
5	202638	3	51618.0	44922.0	58314.0	77.0	67.0	
6	202637	3	37015.0	31541.0	42489.0	55.0	47.0	
7	202636	3	25537.0	19707.0	31367.0	38.0	29.0	
8	202635	3	18039.0	13597.0	22481.0	27.0	20.0	
9	202634	3	12726.0	8755.0	16697.0	19.0	13.0	
10	202633	3	7188.0	3962.0	10414.0	11.0	6.0	
11	202632	3	7870.0	4582.0	11158.0	12.0	7.0	
12	202631	3	7046.0	4169.0	9923.0	10.0	6.0	
13	202630	3	8880.0	5665.0	12095.0	13.0	8.0	
14	202629	3	7601.0	4679.0	10523.0	11.0	7.0	
15	202628	3	9217.0	6138.0	12296.0	14.0	9.0	
16	202627	3	11019.0	7871.0	14167.0	16.0	11.0	
17	202626	3	11341.0	8008.0	14674.0	17.0	12.0	
18	202625	3	12022.0	8825.0	15219.0	18.0	13.0	
19	202624	3	14005.0	10335.0	17675.0	21.0	16.0	
20	202623	3	15536.0	11350.0	19722.0	23.0	17.0	
21	202622	3	11269.0	7864.0	14674.0	17.0	12.0	
22	202621	3	14005.0	10391.0	17619.0	21.0	16.0	
23	202620	3	13818.0	9441.0	18195.0	21.0	15.0	

24	202619		3	10240.0	6610.0	13870.0	15.0	10.0
25	202618		3	11906.0	8174.0	15638.0	18.0	12.0
26	202617		3	14808.0	10600.0	19016.0	22.0	16.0
27	202616		3	21922.0	17127.0	26717.0	33.0	26.0
28	202615		3	21286.0	16456.0	26116.0	32.0	25.0
29	202614		3	29932.0	24335.0	35529.0	44.0	36.0
…	…	…	…	…	…	…	…	
2161	198521		3	26096.0	19621.0	32571.0	47.0	35.0
2162	198520		3	27896.0	20885.0	34907.0	51.0	38.0
2163	198519		3	43154.0	32821.0	53487.0	78.0	59.0
2164	198518		3	40555.0	29935.0	51175.0	74.0	55.0
2165	198517		3	34053.0	24366.0	43740.0	62.0	44.0
2166	198516		3	50362.0	36451.0	64273.0	91.0	66.0
2167	198515		3	63881.0	45538.0	82224.0	116.0	83.0
2168	198514		3	134545.0	114400.0	154690.0	244.0	207.0
2169	198513		3	197206.0	176080.0	218332.0	357.0	319.0
2170	198512		3	245240.0	223304.0	267176.0	445.0	405.0
2171	198511		3	276205.0	252399.0	300011.0	501.0	458.0
2172	198510		3	353231.0	326279.0	380183.0	640.0	591.0
2173	198509		3	369895.0	341109.0	398681.0	670.0	618.0
2174	198508		3	389886.0	359529.0	420243.0	707.0	652.0
2175	198507		3	471852.0	432599.0	511105.0	855.0	784.0
2176	198506		3	565825.0	518011.0	613639.0	1026.0	939.0
2177	198505		3	637302.0	592795.0	681809.0	1155.0	1074.0
2178	198504		3	424937.0	390794.0	459080.0	770.0	708.0
2179	198503		3	213901.0	174689.0	253113.0	388.0	317.0
2180	198502		3	97586.0	80949.0	114223.0	177.0	147.0
2181	198501		3	85489.0	65918.0	105060.0	155.0	120.0
2182	198452		3	84830.0	60602.0	109058.0	154.0	110.0
2183	198451		3	101726.0	80242.0	123210.0	185.0	146.0
2184	198450		3	123680.0	101401.0	145959.0	225.0	184.0
2185	198449		3	101073.0	81684.0	120462.0	184.0	149.0
2186	198448		3	78620.0	60634.0	96606.0	143.0	110.0
2187	198447		3	72029.0	54274.0	89784.0	131.0	99.0
2188	198446		3	87330.0	67686.0	106974.0	159.0	123.0
2189	198445		3	135223.0	101414.0	169032.0	246.0	184.0
2190	198444		3	68422.0	20056.0	116788.0	125.0	37.0

	inc100_up	geo_insee	geo_name
0	110.0	FR	France
1	96.0	FR	France
2	103.0	FR	France
3	105.0	FR	France
4	98.0	FR	France
5	87.0	FR	France
6	63.0	FR	France
7	47.0	FR	France

8	34.0	FR	France
9	25.0	FR	France
10	16.0	FR	France
11	17.0	FR	France
12	14.0	FR	France
13	18.0	FR	France
14	15.0	FR	France
15	19.0	FR	France
16	21.0	FR	France
17	22.0	FR	France
18	23.0	FR	France
19	26.0	FR	France
20	29.0	FR	France
21	22.0	FR	France
22	26.0	FR	France
23	27.0	FR	France
24	20.0	FR	France
25	24.0	FR	France
26	28.0	FR	France
27	40.0	FR	France
28	39.0	FR	France
29	52.0	FR	France
…	…	…	…
2161	59.0	FR	France
2162	64.0	FR	France
2163	97.0	FR	France
2164	93.0	FR	France
2165	80.0	FR	France
2166	116.0	FR	France
2167	149.0	FR	France
2168	281.0	FR	France
2169	395.0	FR	France
2170	485.0	FR	France
2171	544.0	FR	France
2172	689.0	FR	France
2173	722.0	FR	France
2174	762.0	FR	France
2175	926.0	FR	France
2176	1113.0	FR	France
2177	1236.0	FR	France
2178	832.0	FR	France
2179	459.0	FR	France
2180	207.0	FR	France
2181	190.0	FR	France
2182	198.0	FR	France
2183	224.0	FR	France
2184	266.0	FR	France

```
2185      219.0        FR   France
2186      176.0        FR   France
2187      163.0        FR   France
2188      195.0        FR   France
2189      308.0        FR   France
2190      213.0        FR   France

[2190 rows x 10 columns]
```

Nos données utilisent une convention inhabituelle: le numéro de semaine est collé à l'année, donnant l'impression qu'il s'agit de nombre entier. C'est comme ça que Pandas les interprète.

Un deuxième problème est que Pandas ne comprend pas les numéros de semaine. Il faut lui fournir les dates de début et de fin de semaine. Nous utilisons pour cela la bibliothèque `isoweek`.

Comme la conversion des semaines est devenu assez complexe, nous écrivons une petite fonction Python pour cela. Ensuite, nous l'appliquons à tous les points de nos donnés. Les résultats vont dans une nouvelle colonne 'period'.

[7]:
```
def convert_week(year_and_week_int):
    year_and_week_str = str(year_and_week_int)
    year = int(year_and_week_str[:4])
    week = int(year_and_week_str[4:])
    w = isoweek.Week(year, week)
    return pd.Period(w.day(0), 'W')

data['period'] = [convert_week(yw) for yw in data['week']]
```

Il restent deux petites modifications à faire.

Premièrement, nous définissons les périodes d'observation comme nouvel index de notre jeux de données. Ceci en fait une suite chronologique, ce qui sera pratique par la suite.

Deuxièmement, nous trions les points par période, dans le sens chronologique.

[8]:
```
sorted_data = data.set_index('period').sort_index()
```

Nous vérifions la cohérence des données. Entre la fin d'une période et le début de la période qui suit, la différence temporelle doit être zéro, ou au moins très faible. Nous laissons une "marge d'erreur" d'une seconde.

Ceci s'avère tout à fait juste sauf pour deux périodes consécutives entre lesquelles il manque une semaine.

Nous reconnaissons ces dates: c'est la semaine sans observations que nous avions supprimées !

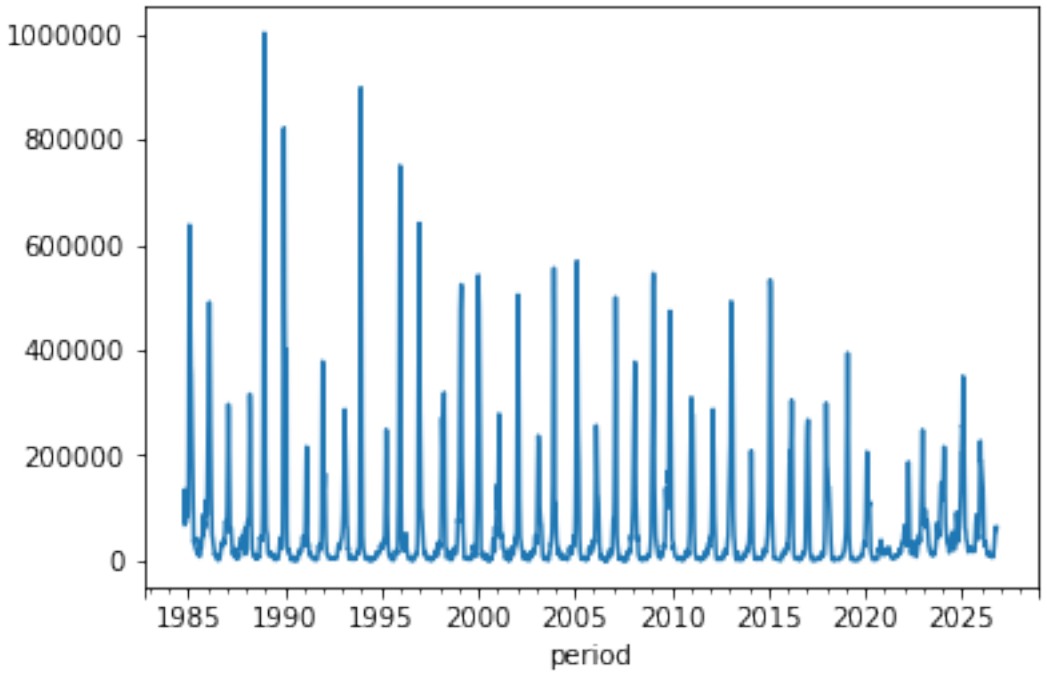
[9]:
```
periods = sorted_data.index
for p1, p2 in zip(periods[:-1], periods[1:]):
    delta = p2.to_timestamp() - p1.end_time
    if delta > pd.Timedelta('1s'):
        print(p1, p2)
```

```
1989-05-01/1989-05-07 1989-05-15/1989-05-21
```

Un premier regard sur les données !

[10]:
```
sorted_data['inc'].plot()
```

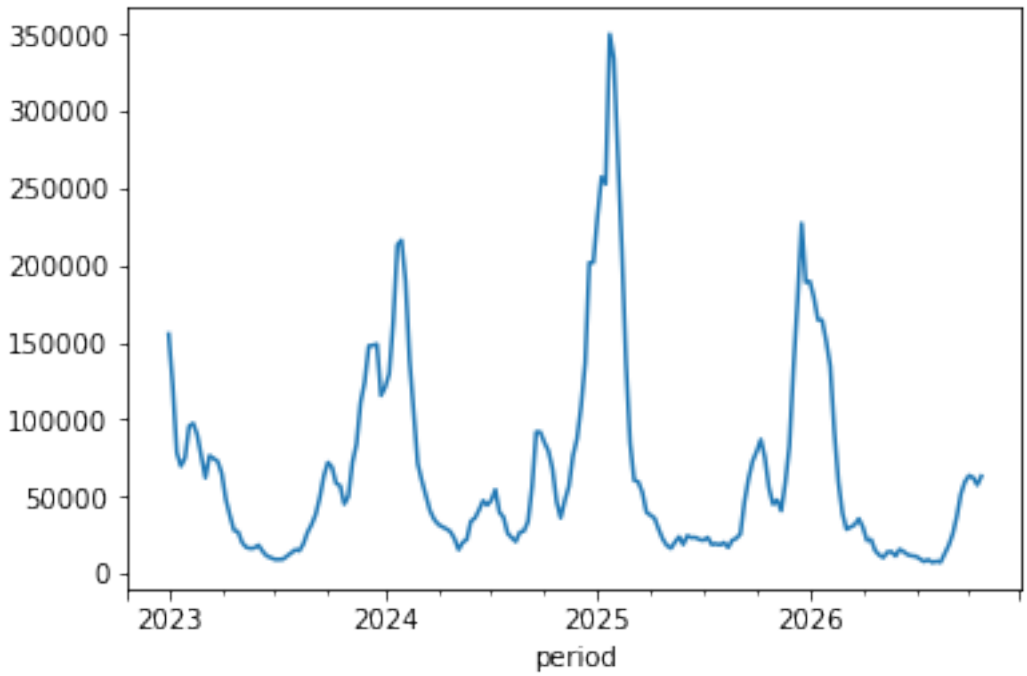
[10]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ddc42e3d518>
```



Un zoom sur les dernières années montre mieux la situation des pics en hiver. Le creux des incidences se trouve en été.

[11]:
```
sorted_data['inc'][-200:].plot()
```

[11]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ddc40bb3668>
```



1.1 Etude de l'incidence annuelle

Etant donné que le pic de l'épidémie se situe en hiver, à cheval entre deux années civiles, nous définissons la période de référence entre deux minima de l'incidence, du 1er août de l'année N au 1er août de l'année $N + 1$.

Notre tâche est un peu compliquée par le fait que l'année ne comporte pas un nombre entier de semaines. Nous modifions donc un peu nos périodes de référence: à la place du 1er août de chaque année, nous utilisons le premier jour de la semaine qui contient le 1er août.

Comme l'incidence de syndrome grippal est très faible en été, cette modification ne risque pas de fausser nos conclusions.

Encore un petit détail: les données commencent an octobre 1984, ce qui rend la première année incomplète. Nous commençons donc l'analyse en 1985.

[12]:
```
first_august_week = [pd.Period(pd.Timestamp(y, 8, 1), 'W')
                     for y in range(1985,
                                    sorted_data.index[-1].year)]
```

En partant de cette liste des semaines qui contiennent un 1er août, nous obtenons nos intervalles d'environ un an comme les périodes entre deux semaines adjacentes dans cette liste. Nous calculons les sommes des incidences hebdomadaires pour toutes ces périodes.

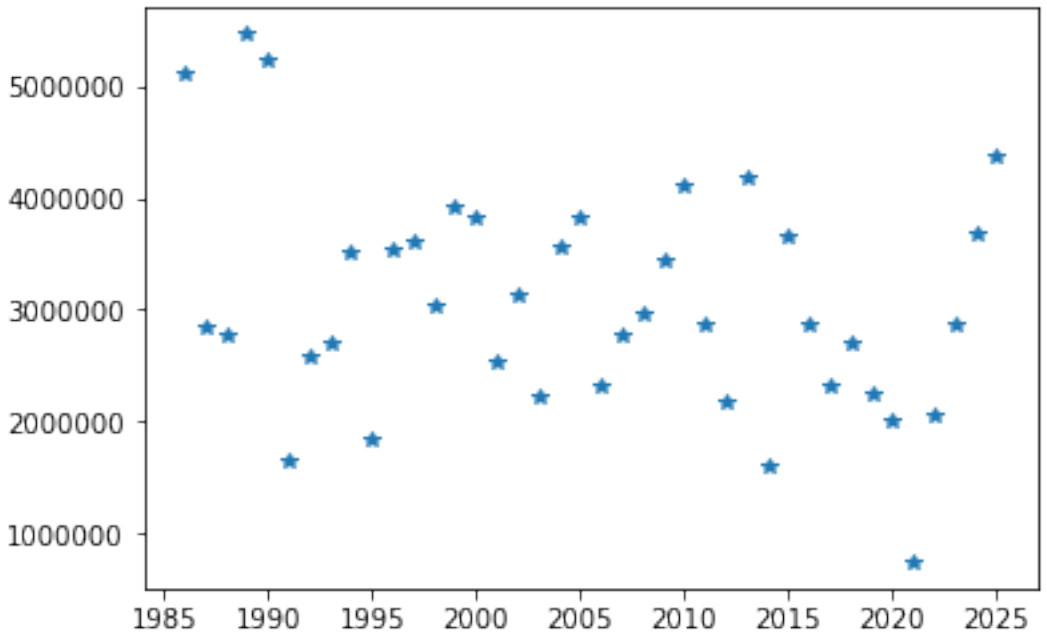
Nous vérifions également que ces périodes contiennent entre 51 et 52 semaines, pour nous protéger contre des éventuelles erreurs dans notre code.

[13]:
```
year = []
yearly_incidence = []
for week1, week2 in zip(first_august_week[:-1],
                        first_august_week[1:]):
    one_year = sorted_data['inc'][week1:week2-1]
    assert abs(len(one_year)-52) < 2
    yearly_incidence.append(one_year.sum())
    year.append(week2.year)
yearly_incidence = pd.Series(data=yearly_incidence, index=year)
```

Voici les incidences annuelles.

[14]:
```
yearly_incidence.plot(style='*')
```

[14]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ddc40ad7438>
```



Une liste triée permet de plus facilement répérer les valeurs les plus élevées (à la fin).

[15]:
```
yearly_incidence.sort_values()
```

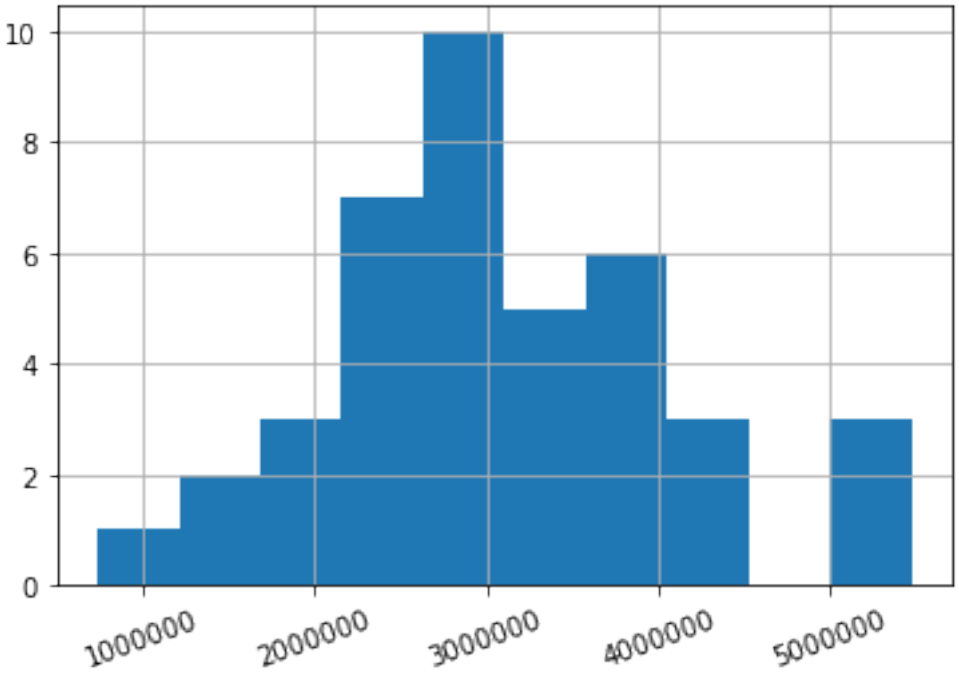
[15]:
```
2021     743449.0
2014    1600941.0
1991    1659249.0
1995    1840410.0
2020    2010315.0
2022    2060304.0
```

```
2012    2175217.0
2003    2234584.0
2019    2254386.0
2006    2307352.0
2017    2321583.0
2001    2529279.0
1992    2574578.0
1993    2703886.0
2018    2705325.0
1988    2765617.0
2007    2780164.0
1987    2855570.0
2016    2856393.0
2011    2857040.0
2023    2873501.0
2008    2973918.0
1998    3034904.0
2002    3125418.0
2009    3444020.0
1994    3514763.0
1996    3539413.0
2004    3567744.0
1997    3620066.0
2015    3654892.0
2024    3670417.0
2000    3826372.0
2005    3835025.0
1999    3908112.0
2010    4111392.0
2013    4182691.0
2025    4366002.0
1986    5115251.0
1990    5235827.0
1989    5466192.0
dtype: float64
```

Enfin, un histogramme montre bien que les épidémies fortes, qui touchent environ 10% de la population française, sont assez rares: il y en eu trois au cours des 35 dernières années.

[16]:
```
yearly_incidence.hist(xrot=20)
```

[16]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ddc40accba8>
```


10
8
6
4
2
0
1000000
2000000
3000000
4000000
5000000

[ ]: